Лабораторная работа №2

дисциплина: Архитектура компьютера

Безходарнова Алиса Викторовна

Содержание

Цель работы 5

Задание 6

Теоретическое введение 7

Выполнение лабораторной работы 9

Вывод 14

Контрольные вопросы 15

Список литературы 18

Список иллюстраций

1 Рис 4.1 Конфигурация git 9
2 Рис 4.2 Создание ssh ключа 9
3 Рис 4.3 Создание gpg ключа 10
4 Рис 4.5 Подключение ключа 11
5 Рис 4.6 Подписи для коммитов 11
6 Рис 4.8 Авторизация на гитхаб 12
7 Рис 4.9 Создание директории курса 12
8 Рис 4.10 Настройка директории 13

Список таблиц

Цель работы

Изучить идеологию и применение средств контроля версий. Освоить умения по работе с git.

Задание

Создать базовую конфигурацию для работы с git. Создать ключ SSH. Создать ключ PGP. Настроить подписи git. Зарегистрироваться на Github. Создать локальный каталог для выполнения заданий по предмету.

Теоретическое введение

Системы контроля версий (Version Control System, VCS) применяются при работе нескольких человек над одним проектом. Обычно основное дерево проекта хранится в локальном или удалённом репозитории, к которому настроен доступ для участников проекта. При внесении изменений в содержание проекта система контроля версий позволяет их фиксировать, совмещать изменения, произведённые разными участниками проекта, производить откат к любой более ранней версии проекта, если это требуется. В классических системах контроля версий используется централизованная модель, предполагающая наличие единого репозитория для хранения файлов. Выполнение большинства функций по управлению версиями осуществляется специальным сервером. Участник проекта (пользователь) перед началом работы посредством определённых команд получает нужную ему версию файлов. После внесения изменений, пользователь размещает новую версию в хранилище. При этом предыдущие версии не удаляются из центрального хранилища и к ним можно вернуться в любой момент. Сервер может сохранять не полную версию изменённых файлов, а производить так называемую дельта-компрессию — сохранять только изменения между последовательными версиями, что позволяет уменьшить объём хранимых данных. Системы контроля версий поддерживают возможность отслеживания и разрешения конфликтов, которые могут возникнуть при работе нескольких человек над одним файлом. Можно объединить (слить) изменения, сделанные разными участниками (автоматически или вручную), вручную выбрать нужную версию, отменить изменения

вовсе или заблокировать файлы для изменения. В зависимости от настроек блокировка не позволяет другим пользователям получить рабочую копию или препятствует изменению рабочей копии файла средствами файловой системы ОС, обеспечивая таким образом, привилегированный доступ только одному пользователю, работающему с файлом. Системы контроля версий также могут обеспечивать дополнительные, более гибкие функциональные возможности.Например, они могут поддерживать работу с несколькими версиями одного файла, сохраняя общую историю изменений до точки ветвления версий и собственные истории изменений каждой ветви. Кроме того, обычно доступна информация о том, кто из участников, когда и какие изменения вносил. Обычно такого рода информация хранится в журнале изменений, доступ к которому можно ограничить. В отличие от классических, в распределённых системах контроля версий центральный репозиторий не является обязательным. Среди классических VCS наиболее известны CVS, Subversion, а среди распределённых — Git, Bazaar, Mercurial. Принципы их работы схожи, отличаются они в основном синтаксисом используемых в работе команд.

Выполнение лабораторной работы

Выполняю базовую настройку git (рис. @fig:git-config).

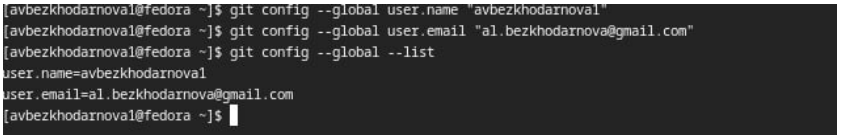


Рис. 1: Рис 4.1 Конфигурация git

Создаю ключ ssh (рис. @fig:ssh).

Рис. 2: Рис 4.2 Создание ssh ключа

Создаю ключ gpg (Рис @fig:gpg).

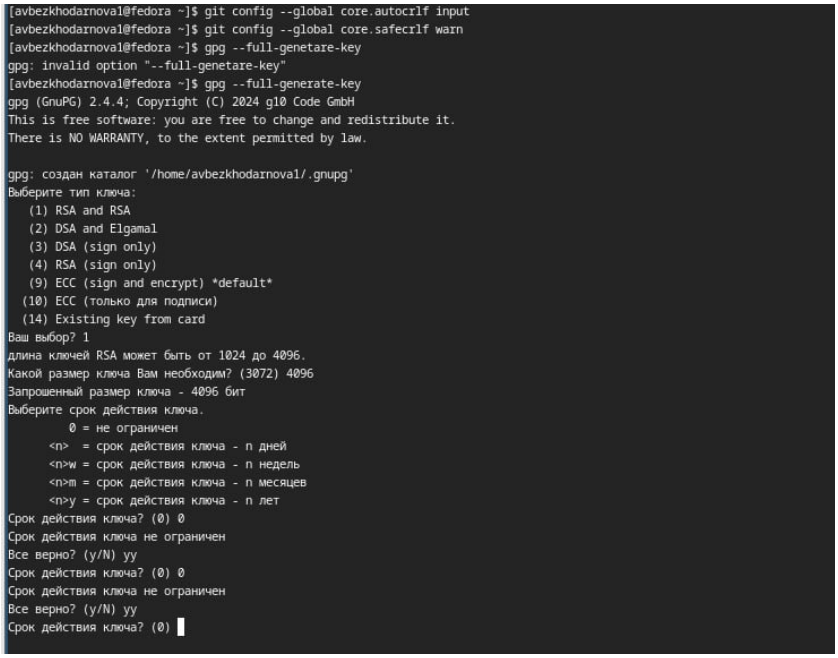
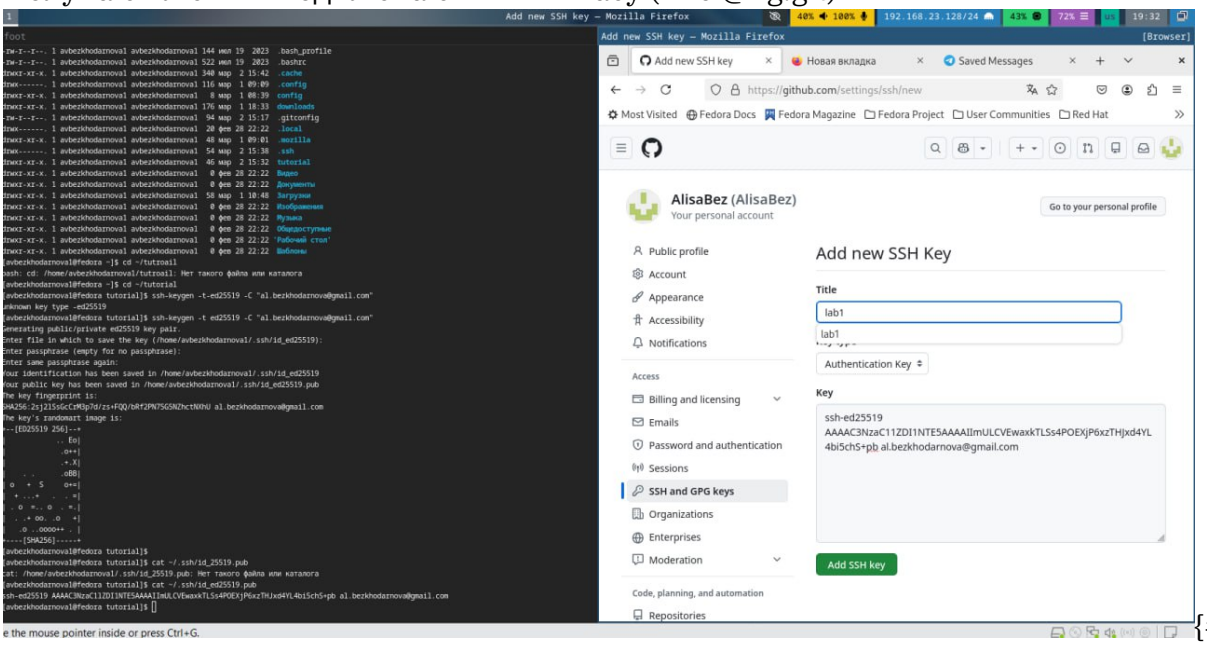


Рис. 3: Рис 4.3 Создание gpg ключа

Получаю ключи и подключаю их к гитхабу (Рис @fig:git)



{#fig:git width=70%)

И (Рис. @fig:gitt)

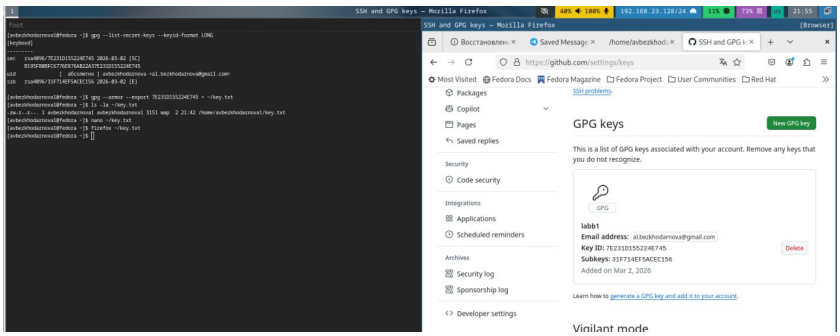
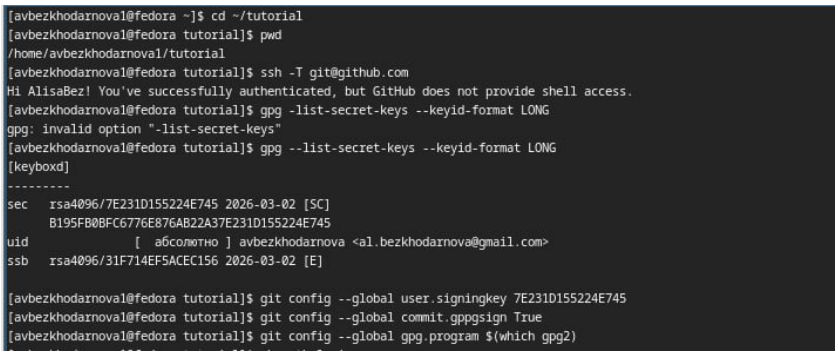


Рис. 4: Рис 4.5 Подключение ключа

Настраиваю автоматические подписи (Рис @fig:006)

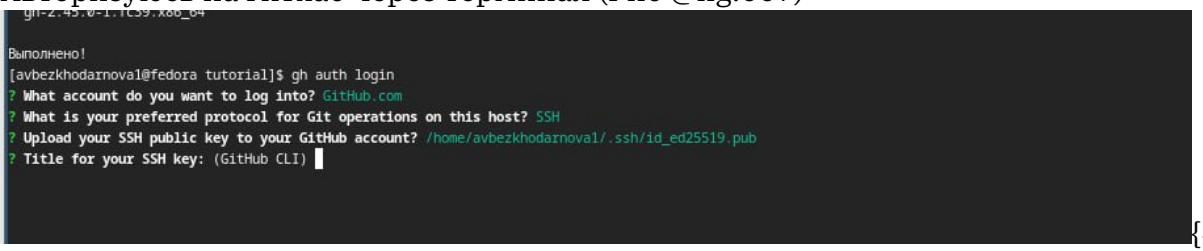


```
[avbezkhodarnova1@fedora ~]$ cd ~/tutorial
[avbezkhodarnova1@fedora tutorial]$ pwd
/home/avbezkhodarnova1/tutorial
[avbezkhodarnova1@fedora tutorial]$ ssh -T git@github.com
Hi AlisaBez! You've successfully authenticated, but GitHub does not provide shell access.
[avbezkhodarnova1@fedora tutorial]$ gpg -list-secret-keys --keyid-format LONG
gpg: invalid option "-list-secret-keys"
[avbezkhodarnova1@fedora tutorial]$ gpg --list-secret-keys --keyid-format LONG
[keyboxd]
---------
sec   rsa4096/7E231D155224E745 2026-03-02 [SC]
      B195FB0BFC6776E876AB22A37E231D155224E745
uid                 [  абсолютно ] avbezkhodarnova <al.bezkhodarnova@gmail.com>
ssb   rsa4096/31F714EF5ACEC156 2026-03-02 [E]

[avbezkhodarnova1@fedora tutorial]$ git config --global user.signingkey 7E231D155224E745
[avbezkhodarnova1@fedora tutorial]$ git config --global commit.gpgsign True
[avbezkhodarnova1@fedora tutorial]$ git config --global gpg.program $(which gpg2)
```

Рис. 5: Рис 4.6 Подписи для коммитов

Авторизуюсь на гитхаб через терминал (Рис @fig:007)



```
Выполнено!
[avbezkhodarnova1@fedora tutorial]$ gh auth login
? What account do you want to log into? GitHub.com
? What is your preferred protocol for Git operations on this host? SSH
? Upload your SSH public key to your GitHub account? /home/avbezkhodarnova1/.ssh/id_ed25519.pub
? Title for your SSH key: (GitHub CLI)
```

{#fig:007 width=70%)

И (Рис @fig:008)

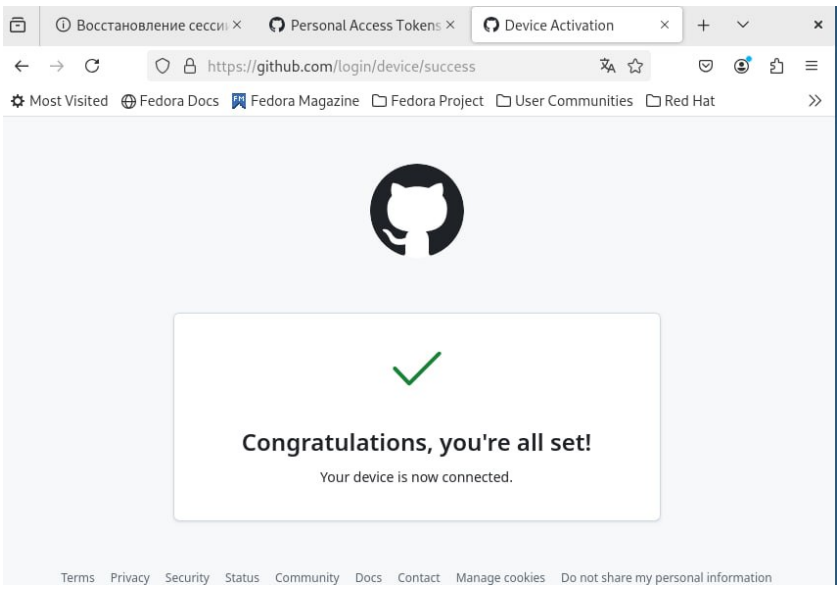


Рис. 6: Рис 4.8 Авторизация на гитхаб

Создаю директорию курса через шаблон (Рис.-@fig:009)

```
! First copy your one-time code: 3994-59D5
Press Enter to open github.com in your browser...
✓ Authentication complete.
- gh config set -h github.com git_protocol ssh
✓ Configured git protocol
✓ SSH key already existed on your GitHub account: /home/avbezkhodarnova1/.ssh/id_ed25519.pub
✓ Logged in as AlisaBez
[avbezkhodarnova1@fedora tutorial]$ ~ cd
bash: /home/avbezkhodarnova1: Это каталог
[avbezkhodarnova1@fedora tutorial]$ cd ~
[avbezkhodarnova1@fedora ~]$ mkdir -p work/study/2025-2026/"oper system"
[avbezkhodarnova1@fedora ~]$ cd  work/study/2025-2026/"oper system"
[avbezkhodarnova1@fedora oper system]$ cd ~/work/study/2025-2026/"oper system"
[avbezkhodarnova1@fedora oper system]$ gh repo create study_2025-2025_os-intro --template=yamadharma/course-directory-student-template --public
✓ Created repository AlisaBez/study_2025-2025_os-intro on GitHub
  https://github.com/AlisaBez/study_2025-2025_os-intro
[avbezkhodarnova1@fedora oper system]$ git clone --recursive git@github.com:<owner>/study_2025-2026_os-intro.git os-intro
bash: owner: Нет такого файла или каталога
[avbezkhodarnova1@fedora oper system]$ git clone --recursive git@github.com:<avbezkhodarnova1>/study_2025-2026_os-intro.git os-intro
bash: avbezkhodarnova1: Нет такого файла или каталога
[avbezkhodarnova1@fedora oper system]$ git clone --recursive git@github.com:<AlisaBez>/study_2025-2026_os-intro.git os-intro
bash: AlisaBez: Нет такого файла или каталога
[avbezkhodarnova1@fedora oper system]$ git clone --recursive git@github.com:AlisaBez/study_2025-2026_os-intro.git os-intro
Клонирование в «os-intro»...
ERROR: Repository not found.
fatal: Не удалось прочитать из внешнего репозитория.

Удостоверьтесь, что у вас есть необходимые права доступа
и репозиторий существует.
[avbezkhodarnova1@fedora oper system]$ git clone --recursive git@github.com:AlisaBez/study_2025-2025_os-intro.git os-intro
Клонирование в «os-intro»...
remote: Enumerating objects: 41, done.
remote: Counting objects: 100% (41/41), done.
remote: Compressing objects: 100% (39/39), done.
remote: Total 41 (delta 1), reused 28 (delta 1), pack-reused 0 (from 0)
Получение объектов: 100% (41/41), 25.54 КиБ | 12.77 МиБ/с, готово.
Определение изменений: 100% (1/1), готово.
Подмодуль «template/presentation» (https://github.com/yamadharma/academic-presentation-markdown-template.git) зарегистрирован по пути «template/presentation»
Подмодуль «template/report» (https://github.com/yamadharma/academic-laboratory-report-template.git) зарегистрирован по пути «template/report»
Клонирование в «/home/avbezkhodarnova1/work/study/2025-2026/oper system/os-intro/template/presentation»...
remote: Enumerating objects: 239, done.
remote: Counting objects: 100% (239/239), done.
remote: Compressing objects: 100% (168/168), done.
remote: Total 239 (delta 96), reused 202 (delta 59), pack-reused 0 (from 0)
Получение объектов: 100% (239/239), 2.67 МиБ | 743.00 КиБ/с, готово.
Определение изменений: 100% (96/96), готово.
Клонирование в «/home/avbezkhodarnova1/work/study/2025-2026/oper system/os-intro/template/report»...
remote: Enumerating objects: 287, done.
remote: Counting objects: 100% (287/287), done.
remote: Compressing objects: 100% (195/195), done.
```

Рис. 7: Рис 4.9 Создание директории курса

И настраиваю рабочую директорию (Рис. @fig:010)

```
remote: Counting objects: 100% (41/41), done.
remote: Compressing objects: 100% (39/39), done.
remote: Total 41 (delta 1), reused 28 (delta 1), pack-reused 0 (from 0)
Получение объектов: 100% (41/41), 25.54 КиБ | 12.77 МиБ/с, готово.
Определение изменений: 100% (1/1), готово.
Подмодуль «template/presentation» (https://github.com/yamadharma/academic-presentation-markdown-template.git) зарегистрирован по пути «template/presentation»
Подмодуль «template/report» (https://github.com/yamadharma/academic-laboratory-report-template.git) зарегистрирован по пути «template/report»
Клонирование в «/home/avbezkhodarnova1/work/study/2025-2026/oper system/os-intro/template/presentation»...
remote: Enumerating objects: 239, done.
remote: Counting objects: 100% (239/239), done.
remote: Compressing objects: 100% (168/168), done.
remote: Total 239 (delta 96), reused 202 (delta 59), pack-reused 0 (from 0)
Получение объектов: 100% (239/239), 2.67 МиБ | 743.00 КиБ/с, готово.
Определение изменений: 100% (96/96), готово.
Клонирование в «/home/avbezkhodarnova1/work/study/2025-2026/oper system/os-intro/template/report»...
remote: Enumerating objects: 287, done.
remote: Counting objects: 100% (287/287), done.
remote: Compressing objects: 100% (195/195), done.
remote: Total 287 (delta 125), reused 234 (delta 72), pack-reused 0 (from 0)
Получение объектов: 100% (287/287), 785.00 КиБ | 1.90 МиБ/с, готово.
Определение изменений: 100% (125/125), готово.
Submodule path 'template/presentation': checked out '60a6091ebedbb01a8d98f2c5fae0f908dc63ad031'
Submodule path 'template/report': checked out '708b8c51acd94d4b78e7634198f221e5decb6355'
[avbezkhodarnova1@fedora oper system]$ cd ~/work/study/2025-2026/"oper system"/os-intro
[avbezkhodarnova1@fedora os-intro]$ rm package.json
[avbezkhodarnova1@fedora os-intro]$ echo os-intro > COURSE
[avbezkhodarnova1@fedora os-intro]$ make
Usage:
  make <target>

Targets:
  list                            List of courses
  prepare                         Generate directories structure
  submodule                       Update submules

[avbezkhodarnova1@fedora os-intro]$ git add .
[avbezkhodarnova1@fedora os-intro]$ git commit -am 'feat(main): make course structure'
[master ad3cd32] feat(main): make course structure
 2 files changed, 1 insertion(+), 14 deletions(-)
 delete mode 100644 package.json
[avbezkhodarnova1@fedora os-intro]$ git push
Перечисление объектов: 5, готово.
Подсчет объектов: 100% (5/5), готово.
При сжатии изменений используется до 2 потоков
Сжатие объектов: 100% (2/2), готово.
Запись объектов: 100% (3/3), 292 байта | 292.00 КиБ/с, готово.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:AlisaBez/study_2025-2025_os-intro.git
   7622560..ad3cd32  master -> master
[avbezkhodarnova1@fedora os-intro]$
```

Рис. 8: Рис 4.10 Настройка директории

Вывод

В ходе данной лабораторной работы я изучала идеологию и применение средств контроля версий, а также освоила работу с git

Контрольные вопросы

1. Что такое системы контроля версий (VCS) и для решения каких задач они предназначаются? Системы контроля версий применяются при работе нескольких человек над одним проектом. Они позволяют фиксировать изменения, совмещать изменения разных участников, производить откат к любой более ранней версии проекта.
2. Объясните следующие понятия VCS и их отношения: хранилище, commit, история, рабочая копия. Хранилище — место хранения всех файлов проекта и истории изменений (в Git это папка .git). Commit — операция сохранения изменений в репозитории. История — последовательность коммитов, отражающая все изменения проекта. Рабочая копия — файлы проекта на локальном компьютере, с которыми непосредственно работает пользователь.
3. Что представляют собой и чем отличаются централизованные и децентрализованные VCS? Приведите примеры VCS каждого вида. В классических системах используется централизованная модель, предполагающая наличие единого репозитория для хранения файлов. Выполнение большинства функций по управлению версиями осуществляется специальным сервером. Примеры: CVS, Subversion. В отличие от классических, в распределённых системах контроля версий центральный репозиторий не является обязательным. Примеры: Git, Bazaar, Mercurial.
4. Опишите действия с VCS при единоличной работе с хранилищем. Инициализация репозитория (git init), создание и изменение файлов, добавление

их в индекс (git add), фиксация изменений (git commit).

5. Опишите порядок работы с общим хранилищем VCS. Получение актуальной версии из центрального репозитория (git pull), создание новой ветки (git checkout -b имя_ветки), внесение изменений, добавление (git add) и фиксация (git commit), отправка изменений в центральный репозиторий (git push origin имя_ветки).
6. Каковы основные задачи, решаемые инструментальным средством git? Управление версиями, обеспечение совместной работы над проектом, возможность отката к любой предыдущей версии, поддержка ветвления и слияния.
7. Назовите и дайте краткую характеристику командам git. git init — создание нового репозитория

git add — добавление изменений в индекс

git commit — фиксация изменений

git push — отправка изменений в удалённый репозиторий

git pull — получение изменений из удалённого репозитория

git status — просмотр состояния файлов

git log — просмотр истории коммитов

git checkout — переключение между ветками

git branch — управление ветками

git merge — слияние веток 8. Приведите примеры использования при работе с локальным и удалённым репозиториями. Локальный репозиторий: mkdir tutorial; cd tutorial; git init; echo ‘hello world’ > hello.txt; git add hello.txt; git commit -am ‘Новый файл’ Удалённый репозиторий: git remote add origin ssh://git@github.com/username/reponame.git; git push -u origin master 9. Что такое и зачем могут быть нужны ветви (branches)? Ветви позволяют вести параллельную разработку, изолировать изменения, экспериментировать, не затрагивая основную версию проекта, а затем объединять изменения через слияние. 10. Как и зачем можно игнорировать некоторые файлы при commit? Для этого

создаётся файл .gitignore, в который прописываются шаблоны файлов и каталогов (например, *.tmp,* .o, build/), которые не должны попадать в репозиторий. Это позволяет исключить временные файлы, объектные файлы, создаваемые компиляторами, и другие служебные данные.

Список литературы